

pandas 데이터 구조 #1. Series

```
In [1]: import numpy as np
import pandas as pd
```

```
In [2]: print(f'numpy:{np.__version__}, pandas:{pd.__version__}')
```

numpy:2.5.2, pandas:3.0.5

설정변경코드

- 변수 명이 두 번 이상 출력되어도 모두 콘솔에서 보여줄 것

```
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity="all"
InteractiveShell.ast_node_interactivity : 'all' | 'last' | 'last_expr' |
'none' (기본값은 'last_expr')
```

```
In [6]: # from IPython.core.interactiveshell import InteractiveShell
# InteractiveShell.ast_node_interactivity="last_expr"
```

```
In [9]: # a=3
# b=5
# print(a,b)
# a+b
```

학습내용

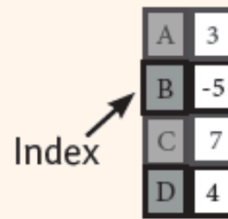
1. 시리즈 소개
2. 시리즈 생성
3. 시리즈 속성들
4. 시리즈 인덱싱
5. 시리즈 슬라이싱
6. 시리즈 연산
7. 시리즈 데이터의 갱신, 추가, 삭제
8. 시리즈 관련 함수들
9. 날짜 인덱스 생성

1. Series 소개

- pandas의 기본 객체 중 하나
- numpy의 ndarray를 기반으로 **인덱싱 기능**을 추가하여 **1차원 배열**을 나타냄
- index를 지정하지 않으며 기본적으로 ndarray와 같이 0-based 인덱스 생성
- index를 지정할 경우 명시적으로 지정된 index를 사용
- 같은 타입의 0개 이상의 데이터를 가질 수 있음

Series

A one-dimensional labeled array
capable of holding any data type



A	3
B	-5
C	7
D	4

```
>>> s = pd.Series([3, -5, 7, 4], index=['a', 'b', 'c', 'd'])
```

- 출처 : <https://www.kdnuggets.com/2017/01/pandas-cheat-sheet.html>

시리즈의 자료구조

- 데이터가 순차적으로 나열된 1차원 배열 형태
- 인덱스(index)와 데이터 값(value)이 일대일로 대응
 - 딕셔너리와 비슷한 구조 : {key(index):value}

시리즈의 인덱스

- 데이터 값의 위치를 나타내는 이름표 역할
- 데이터 값의 순서와 주소를 저장
- 데이터 값의 탐색, 정렬, 선택, 결합 등 데이터 조작을 쉽게 할 수 있음

시리즈 생성 : 판다스 내장함수인 Series()이용

- 리스트, 딕셔너리, 튜플 등으로 시리즈 생성

2. Series 생성하기

pd.Series(data=seq_data, index=None, dtype=None, name=None,...) 함수 이용

- seq_data : array-like, Iterable, dict, or scalar value
- index : array-like or Index(1d)
- dtype : str, numpy.dtype, or ExtensionDtype
- name : str, optional (시리즈의 이름)
- API
 - <https://pandas.pydata.org/docs/reference/api/pandas.Series.html>
 - <https://pandas.pydata.org/docs/reference/series.html>

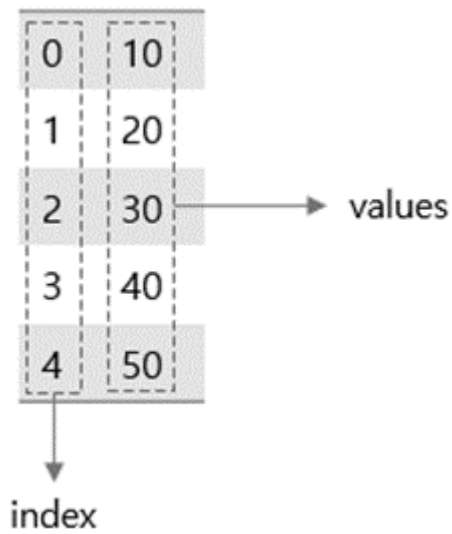
1) seq_data를 이용하여 Series 생성

- index를 지정하지 않아도 자동적으로 생성됨 : 0부터 숫자로 생성됨

① 리스트로 시리즈 만들기

```
In [10]: # 인덱스 없이 시리즈 생성 시 자동으로 정수(위치) 인덱스 생성됨
s = pd.Series([10, 20, 30, 40, 50])
s
```

```
Out[10]: 0    10
         1    20
         2    30
         3    40
         4    50
dtype: int64
```



```
In [11]: # 정수값을 갖은 리스트로 시리즈 생성
pd.Series([11,22,33,44,55], dtype='int32')
```

```
Out[11]: 0    11
         1    22
         2    33
         3    44
         4    55
dtype: int32
```

```
In [12]: # 문자열을 갖은 리스트로 시리즈 생성
pd.Series(['a','b','c'])
```

```
Out[12]: 0    a
         1    b
         2    c
dtype: str
```

```
In [14]: pd.Series('apple kiwi mango'.split())
```

```
Out[14]: 0    apple
         1    kiwi
         2    mango
dtype: str
```

② 정수 범위 자료를 시리즈로 생성

- range() / np.arange() 함수 사용

```
In [15]: # range() 함수를 이용하여 시리즈 생성
pd.Series(range(10,16))
```

```
Out[15]: 0    10
         1    11
         2    12
         3    13
         4    14
         5    15
dtype: int64
```

```
In [16]: # numpy.arange() 함수를 이용하여 시리즈 생성
pd.Series(np.arange(10,16), dtype='int32')
```

```
Out[16]: 0    10
         1    11
         2    12
         3    13
         4    14
         5    15
dtype: int32
```

③ 결측값을 포함한 시리즈 생성

- 결측값 NaN : numpy 모듈 nan 사용
- 결측값 생성을 위해 numpy 모듈 임포트

```
In [19]: pd.Series((1,2, np.nan, 3,4,5))
```

```
Out[19]: 0    1.0
         1    2.0
         2    NaN
         3    3.0
         4    4.0
         5    5.0
dtype: float64
```

④ 딕셔너리로 시리즈 생성

- **Series({key1:value1, key2:value2, key3:value3, ... })**
 - 인덱스 -> key
 - 값 -> value

```
In [20]: data = {'홍길동':30, '강감찬':20, '이순신':40}
pd.Series(data)
```

```
Out[20]: 홍길동    30
         강감찬    20
         이순신    40
dtype: int64
```

2) 인덱스를 명시하여 시리즈 생성

[형식] pd.Series([값1, 값2, 값3, ...], index=[1,2,3, ...])

① 숫자 인덱스 지정

- s = pd.Series([값1, 값2, 값3, ...], index=[1,2,3, ...])

```
In [21]: # 숫자(정수, 위치) 인덱스 지정
pd.Series(data=[10,20,30,40], index=[1,2,3,4])
```

```
Out[21]: 1    10
         2    20
         3    30
         4    40
dtype: int64
```

② 문자 인덱스 지정

```
In [22]: # 문자 인덱스 지정
pd.Series(data=[40,30,50,30], index=['Kim','Lee','Choi','Kang'])
```

```
Out[22]: Kim      40
Lee       30
Choi     50
Kang     30
dtype: int64
```

③ 딕셔너리 기반으로 생성한 시리즈 인덱스 설정

- 딕셔너리의 원소는 순서를 갖지 않는다.
 - 딕셔너리로 생성된 시리즈의 원소도 순서가 보장되지 않는다.
 - 만약 순서를 보장하고 싶으면 인덱스를 리스트로 지정해야 한다.

```
In [23]: city = {'Seoul':99000000, 'Pusan':34000000, 'Incheon':27000000, 'Daejeon':15000000}
pd.Series(data=city)
```

```
Out[23]: Seoul      99000000
Pusan       34000000
Incheon     27000000
Daejeon     15000000
dtype: int64
```

```
In [26]: pd.Series(data=city.values(), index=['서울','부산','인천','대전'])
```

```
Out[26]: 서울      99000000
부산       34000000
인천       27000000
대전       15000000
dtype: int64
```

3. 시리즈의 속성들

1) 시리즈의 index

- 시리즈의 index는 index 속성으로 접근 : **시리즈.index**

```
In [29]: city = {'Seoul':99000000, 'Pusan':34000000, 'Incheon':27000000, 'Daejeon':15000000}
s = pd.Series(data=city)
```

```
In [28]: s.index
```

```
Out[28]: Index(['Seoul', 'Pusan', 'Incheon', 'Daejeon'], dtype='str')
```

```
In [30]: type(s.index)
```

```
Out[30]: pandas.Index
```

2) 시리즈.index.name 속성

- 시리즈의 인덱스에 이름을 붙일 수 있음

```
In [32]: print(s.index.name)
```

None

```
In [33]:
```

```
s
```

```
Out[33]:
```

```
Seoul      99000000  
Pusan      34000000  
Incheon    27000000  
Daejeon    15000000  
dtype: int64
```

```
In [34]:
```

```
s.index.name = 'Area'
```

```
In [35]:
```

```
s
```

```
Out[35]:
```

```
Area  
Seoul      99000000  
Pusan      34000000  
Incheon    27000000  
Daejeon    15000000  
dtype: int64
```

3) 시리즈의 값(value)

- numpy 자료구조 : 1차원 배열
- values 속성으로 접근 : **시리즈.values**

```
In [36]:
```

```
s
```

```
Out[36]:
```

```
Area  
Seoul      99000000  
Pusan      34000000  
Incheon    27000000  
Daejeon    15000000  
dtype: int64
```

```
In [38]:
```

```
s.values # numpy array
```

```
Out[38]:
```

```
array([99000000, 34000000, 27000000, 15000000])
```

```
In [39]:
```

```
type(s.values)
```

```
Out[39]:
```

```
numpy.ndarray
```

4) 시리즈.name 속성

- 시리즈 데이터에 이름을 붙일 수 있음
- name 속성은 값의 의미 전달에 사용

```
In [41]:
```

```
print(s.name)
```

None

```
In [42]:
```

```
s.name = 'population'
```

```
In [43]:
```

```
s
```

```
Out[43]:
```

```
Area  
Seoul      99000000  
Pusan      34000000  
Incheon    27000000  
Daejeon    15000000  
Name: population, dtype: int64
```

```
In [44]: s.name
```

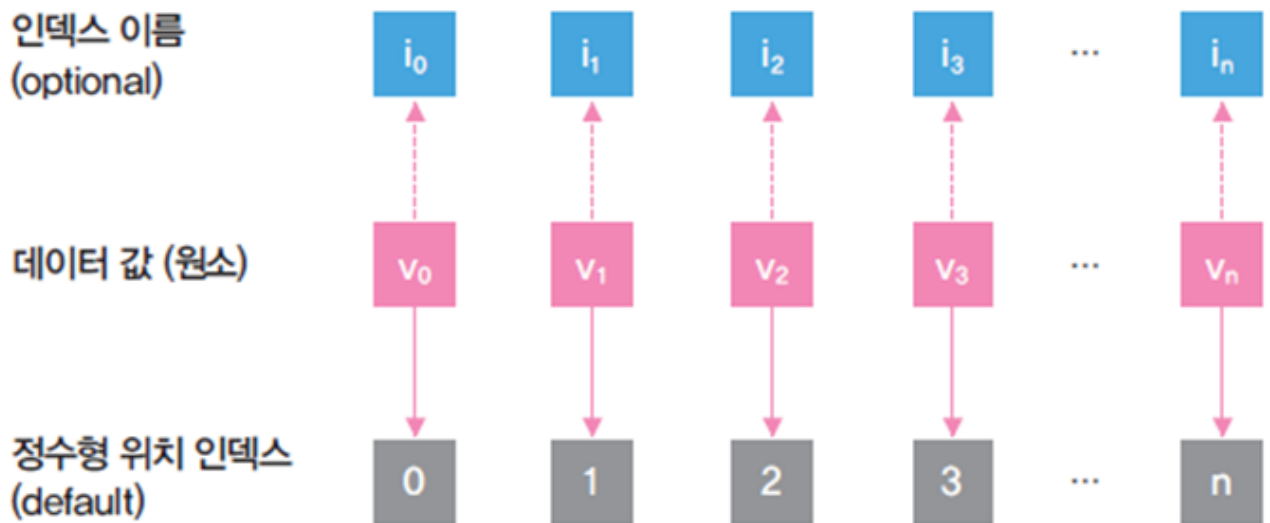
```
Out[44]: 'population'
```

4. 시리즈 인덱싱(indexing)

시리즈의 인덱싱 종류

- 정수형 위치 인덱스(integer position)
 - RangeIndex 객체
- 인덱스 이름(index name) 또는 인덱스 라벨(index label)

<인덱스 구조>



※인덱스를 지정하지 않으면 0부터 시작하는 정수형 인덱스가 지정됨

1) 원소 접근

- 정수형 위치인덱스 : 객체명[정수인덱스]
 - 예. `s[0]`
- 문자형 인덱스 : 객체명['문자인덱스'] 또는 객체명.문자인덱스명
 - 예. `s['인천']`, `s.인천`

① 정수형 위치인덱스 접근

```
In [46]: s1 = pd.Series(data=[10,20,30,40])
s1
```

```
Out[46]: 0    10
         1    20
         2    30
         3    40
         dtype: int64
```

```
In [47]: s1.index
```

```
Out[47]: RangeIndex(start=0, stop=4, step=1)
```

```
In [49]: print(s1[2])
```

30

② 문자형 인덱스 접근

```
In [51]: s
```

```
Out[51]: Area
Seoul      99000000
Pusan      34000000
Incheon    27000000
Daejeon    15000000
Name: population, dtype: int64
```

```
In [52]: s.index
```

```
Out[52]: Index(['Seoul', 'Pusan', 'Incheon', 'Daejeon'], dtype='str', name='Area')
```

```
In [53]: for idx in s.index:
          print(f'index={idx}, value={s[idx]}')
```

```
index=Seoul, value=99000000
index=Pusan, value=34000000
index=Incheon, value=27000000
index=Daejeon, value=15000000
```

```
In [54]: # 문제. s 시리즈에서 부산의 값을 가져오기
s['Pusan']
```

```
Out[54]: np.int64(34000000)
```

```
In [55]: s.Pusan
```

```
Out[55]: np.int64(34000000)
```

```
In [57]: # 문자열 인덱스를 갖는 시리즈에 대해 정수번째 값 가져오기
s.iloc[2] # iloc : integer location -> n번째 위치 인덱스
```

```
Out[57]: np.int64(27000000)
```

2) 리스트 이용 인덱싱

- 자료의 순서를 바꾸거나 특정자료 여러 개를 선택할 수 있다.
- 인덱스값 여러 개를 이용해 접근시 []안에 넣는다

```
In [59]: # 여러 개 접근
# [정수인덱스1, 정수인덱스2, ...]
s1[[1,3]]
```

```
Out[59]: 1      20
          3      40
          dtype: int64
```

```
In [60]: # [문자인덱스1, 문자인덱스2, ...]
s[['Pusan', 'Daejeon']]
```

```
Out[60]: Area
Pusan      34000000
Daejeon    15000000
Name: population, dtype: int64
```


3) 인덱스 통한 시리즈 데이터 변경

In [61]:

```
s
```

Out[61]:

```
Area
Seoul      99000000
Pusan      34000000
Incheon    27000000
Daejeon    15000000
Name: population, dtype: int64
```

In [64]: *# '서울' 데이터 변경*

```
print(s['Seoul'])
print(s.Seoul)
s['Seoul'] = 99100000
s
```

```
99000000
```

```
99000000
```

Out[64]:

```
Area
Seoul      99100000
Pusan      34000000
Incheon    27000000
Daejeon    15000000
Name: population, dtype: int64
```

4) 인덱스 재사용

In [65]:

```
s.index
```

Out[65]: Index(['Seoul', 'Pusan', 'Incheon', 'Daejeon'], dtype='str', name='Area')

In [66]:

```
pd.Series(data=[11320036, 3567382, 2801394, 1568900],
          index=s.index)
```

Out[66]:

```
Area
Seoul      11320036
Pusan       3567382
Incheon     2801394
Daejeon     1568900
dtype: int64
```

5. 시리즈 슬라이싱(slicing)

- 정수형 위치 인덱스를 사용한 슬라이싱
 - 시리즈[start : stop+1]
- 문자(라벨)인덱스 이용 슬라이싱
 - 시리즈['시작라벨' : '끝라벨'] : 표시된 라벨 범위 모두 추출

In [69]: *# 정수형 위치 인덱스를 사용하여 슬라이싱*

```
s2 = pd.Series([10, 20, 30, 40, 50, 60, 70])
print(s2[2:5])
type(s2[2:5])
```

```
2    30
```

```
3    40
```

```
4    50
```

```
dtype: int64
```

```
Out[69]: pandas.Series
```

```
In [70]: s2[1::2]
```

```
Out[70]: 1    20
         3    40
         5    60
         dtype: int64
```

```
In [71]: # 문자열인덱스를 사용하여 슬라이싱
s
```

```
Out[71]: Area
Seoul    99100000
Pusan    34000000
Incheon  27000000
Daejeon  15000000
Name: population, dtype: int64
```

```
In [73]: s['Pusan':'Daejeon']
```

```
Out[73]: Area
Pusan    34000000
Incheon  27000000
Daejeon  15000000
Name: population, dtype: int64
```

6. 시리즈 연산

1) 벡터화 연산

- numpy 배열처럼 pandas의 시리즈도 벡터화 연산 가능
- 벡터화 연산이란 집합적 자료형의 원소 각각을 독립적으로 계산을 진행하는 방법
 - 단, 연산은 시리즈의 값에만 적용되며 인덱스 값은 변경 불가

시리즈와 숫자(스칼라) 연산 : Series객체 + 연산자(+, *, -, /) + 숫자

```
In [74]: s = pd.Series(data={'홍길동':30, '강감찬':20, '이순신':40})
s
```

```
Out[74]: 홍길동    30
         강감찬    20
         이순신    40
         dtype: int64
```

```
In [75]: # 시리즈와 스칼라의 합
s + 5
```

```
Out[75]: 홍길동    35
         강감찬    25
         이순신    45
         dtype: int64
```

```
In [76]: s = pd.Series(data=city)
s
```

```
Out[76]: Seoul    99000000
         Pusan    34000000
         Incheon  27000000
         Daejeon  15000000
         dtype: int64
```

```
In [78]: # 시리즈의 값을 1/1000000로 변경  
s / 1000000
```

```
Out[78]: Seoul      9.9  
Pusan       3.4  
Incheon     2.7  
Daejeon     1.5  
dtype: float64
```

2) 벡터화 인덱싱(Boolean selection)

- boolean Series가 []와 함께 사용되면 True 값에 해당하는 값만 새로 반환되는 Series객체에 포함됨
- 다중조건인 경우, &(and), |(or)를 사용하여 연결 가능

```
In [79]: s
```

```
Out[79]: Seoul      99000000  
Pusan      34000000  
Incheon    27000000  
Daejeon    15000000  
dtype: int64
```

```
In [80]: s > 30000000
```

```
Out[80]: Seoul      True  
Pusan      True  
Incheon    False  
Daejeon    False  
dtype: bool
```

```
In [81]: s[s > 30000000]
```

```
Out[81]: Seoul      99000000  
Pusan      34000000  
dtype: int64
```

```
In [83]: s[s<60000000]
```

```
Out[83]: Pusan      34000000  
Incheon    27000000  
Daejeon    15000000  
dtype: int64
```

```
In [84]: (s>30000000) & (s<50000000)
```

```
Out[84]: Seoul      False  
Pusan      True  
Incheon    False  
Daejeon    False  
dtype: bool
```

```
In [85]: s[(s>30000000) & (s<50000000)]
```

```
Out[85]: Pusan      34000000  
dtype: int64
```

```
In [87]: s = pd.Series(np.arange(1,15,3))  
s
```

```
Out[87]: 0      1
          1      4
          2      7
          3     10
          4     13
          dtype: int64
```

```
In [88]: # 값이 5보다 큰 요소 추출
s[s > 5]
```

```
Out[88]: 2      7
          3     10
          4     13
          dtype: int64
```

```
In [90]: # 짝수 추출
s[s % 2 == 0]
```

```
Out[90]: 1      4
          3     10
          dtype: int64
```

```
In [94]: # 인덱스가 3 이상인 요소들 추출
s[s.index >= 3]
```

```
Out[94]: 3     10
          4     13
          dtype: int64
```

```
In [97]: # 값이 5 이상 8 이하인 요소 추출
s[(s >= 5) & (s <= 8)]
```

```
Out[97]: 2      7
          dtype: int64
```

```
In [100... # 값이 7 이상인 수들의 합
s[s >= 7].sum()
```

```
Out[100... np.int64(30)
```

```
In [101... # 값이 7 이상인 수들의 평균
s[s >= 7].mean()
```

```
Out[101... np.float64(10.0)
```

```
In [102... # 값이 7 이상인 수들의 개수
s[s >= 7].count()
```

```
Out[102... np.int64(3)
```

```
In [106... from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity="all"
```

3) 두 시리즈 간의 연산

시리즈와 시리즈 연산 : Series1 + 연산자(+, *, -, /) + Series2

- 시리즈간의 연산은 같은 인덱스를 찾아 연산을 진행
- 동일한 인덱스는 연산을 진행하고 나머지 인덱스는 연산처리가 불가 해 NaN 값 처리

```
In [107...] s1 = pd.Series(data=[10,20,15,25], index='a b c d'.split())
s2 = pd.Series(data=[5,6,7,8], index='b c d a'.split())
s1
s2
```

```
Out[107...] a    10
b    20
c    15
d    25
dtype: int64
```

```
Out[107...] b     5
c     6
d     7
a     8
dtype: int64
```

① 시리즈 + 시리즈

- 같은 인덱스를 찾아 + 연산을 진행

```
In [108...] # 두 시리즈에 대한 + 연산 : 같은 인덱스를 기준으로 + 연산 수행
s1 + s2
```

```
Out[108...] a    18
b    25
c    21
d    32
dtype: int64
```

- 동일한 인덱스가 없는 경우 나머지 인덱스는 연산처리가 불가하여 NaN으로 처리

```
In [112...] s3 = pd.Series(data=[10,9,11,14], index='e d f g'.split())
s4 = pd.Series(data=[-4,2,-6,7], index='a b e d'.split())
s5 = pd.Series(data=[10,12,np.nan,5], index='a b e d'.split())
s3
s4
s5
```

```
Out[112...] e    10
d     9
f    11
g    14
dtype: int64
```

```
Out[112...] a    -4
b     2
e    -6
d     7
dtype: int64
```

```
Out[112...] a    10.0
b    12.0
e     NaN
d     5.0
dtype: float64
```

```
In [110...] s1 + s3
```

```
Out[110...] a      NaN
             b      NaN
             c      NaN
             d    34.0
             e      NaN
             f      NaN
             g      NaN
             dtype: float64
```

```
In [111...] s3 + s4
```

```
Out[111...] a      NaN
             b      NaN
             d    16.0
             e     4.0
             f      NaN
             g      NaN
             dtype: float64
```

```
In [113...] s4+s5
```

```
Out[113...] a     6.0
             b    14.0
             e      NaN
             d    12.0
             dtype: float64
```

② 시리즈 - 시리즈

```
In [114...] s1 - s2
```

```
Out[114...] a     2
             b    15
             c     9
             d    18
             dtype: int64
```

```
In [115...] s1 - s4
```

```
Out[115...] a    14.0
             b    18.0
             c     NaN
             d    18.0
             e     NaN
             dtype: float64
```

```
In [119...] # 참고. >, >=, <, <=
s1
s2
s3
s4
s5
# s1 > s2
# s1 < s5
s4 > s5
s4[s4 > s5]
```

```
Out[119...] a    10
             b    20
             c    15
             d    25
             dtype: int64
```

```
Out[119... b    5
             c    6
             d    7
             a    8
             dtype: int64
```

```
Out[119... e    10
             d     9
             f    11
             g    14
             dtype: int64
```

```
Out[119... a    -4
             b     2
             e    -6
             d     7
             dtype: int64
```

```
Out[119... a    10.0
             b    12.0
             e     NaN
             d     5.0
             dtype: float64
```

```
Out[119... a    False
             b    False
             e    False
             d     True
             dtype: bool
```

```
Out[119... d     7
             dtype: int64
```

③ 시리즈의 values 속성을 이용한 연산

- values 속성을 사용하면 시리즈의 형태가 사라지므로 동일 위치 원소들끼리 연산
- 시리즈.values 는 array 형태 반환

```
In [120... # 시리즈의 values 속성은 ndarray 형태로 가져옴
s1
s1.values
s2
s2.values
```

```
Out[120... a    10
             b    20
             c    15
             d    25
             dtype: int64
```

```
Out[120... array([10, 20, 15, 25])
```

```
Out[120... b     5
             c     6
             d     7
             a     8
             dtype: int64
```

```
Out[120... array([5, 6, 7, 8])
```

```
In [121... # 시리즈의 요소간에 - 연산 수행 : 인덱스 무시
s1.values * s2.values
```

```
Out[121... array([ 50, 120, 105, 200])
```

4) 시리즈 산술연산 메서드

- Series1.add(Series2)

- Series1.sub(Series2)
- Series1.mul(Series2)
- Series1.div(Series2)

```
In [122... s1.add(s2)
s4.sub(s5)
s1.mul(s4)
s1.div(s2)
```

```
Out[122... a    18
b    25
c    21
d    32
dtype: int64
```

```
Out[122... a   -14.0
b   -10.0
e      NaN
d     2.0
dtype: float64
```

```
Out[122... a   -40.0
b    40.0
c      NaN
d   175.0
e      NaN
dtype: float64
```

```
Out[122... a    1.250000
b    4.000000
c    2.500000
d    3.571429
dtype: float64
```

5) in 연산자

- 시리즈 객체는 라벨(문자)에 의해 인덱싱이 가능
- 실질적으로는 라벨을 key로 가지는 딕셔너리 형과 같다고 볼 수 있음
- 딕셔너리에서 제공하는 대부분의 연산자 사용 가능
 - in 연산자 : True / False
 - for 루프를 통해 각 원소의 key와 value에 접근

```
In [123... city = {'Seoul': 9731282, 'Pusan': 3393191, 'Incheon': 2632035, 'Daejeon': 1490158}
s = pd.Series(data=city)
s
s.index = ['서울', '부산', '인천', '대전']
s
```

```
Out[123... Seoul      9731282
Pusan       3393191
Incheon     2632035
Daejeon     1490158
dtype: int64
```

```
Out[123... 서울      9731282
부산       3393191
인천       2632035
대전       1490158
dtype: int64
```

```
In [124... # 인덱스가 '서울'인 요소가 시리즈에 있는지?
'서울' in s
```


Out[124... True

```
In [125... # 인덱스가 '대전'인 요소가 시리즈에 있는가?
'대전' in s
```

Out[125... True

```
In [126... # 인덱스가 '대구'인 요소가 시리즈에 있는가?
'대구' in s
```

Out[126... False

```
In [127... # 인덱스가 '인천'인 요소가 시리즈에 없는가?
'인천' not in s
```

Out[127... False

```
In [129... # 딕셔너리의 items()와 같은 함수를 시리즈에서도 제공 : 키와 값
# items()는 리스트로 변환하여 사용
list(s.items())
```

Out[129... [('서울', 9731282), ('부산', 3393191), ('인천', 2632035), ('대전', 1490158)]

```
In [130... # 시리즈의 각 요소의 인덱스와 값을 출력
for k,v in s.items():
    print(f'{k}:{v}')
```

서울:9731282

부산:3393191

인천:2632035

대전:1490158

```
In [133... s.keys()
s.index
```

Out[133... Index(['서울', '부산', '인천', '대전'], dtype='str')

Out[133... Index(['서울', '부산', '인천', '대전'], dtype='str')

```
In [135... s.values
```

Out[135... array([9731282, 3393191, 2632035, 1490158])

7. 시리즈 데이터의 갱신, 추가, 삭제

인덱싱을 이용하면 딕셔너리 처럼 갱신, 추가 가능

```
In [136... s
```

Out[136... 서울 9731282
부산 3393191
인천 2632035
대전 1490158
dtype: int64

1) 시리즈 데이터 갱신

```
In [138... s['부산']=3500000
s
```

```
Out[138... 서울    9731282
부산    3500000
인천    2632035
대전    1490158
dtype: int64
```

```
In [139... s.부산 = 3393191
s
```

```
Out[139... 서울    9731282
부산    3393191
인천    2632035
대전    1490158
dtype: int64
```

2) 시리즈 데이터 삭제

```
In [140... # 딕셔너리에서 요소 삭제를 위해 사용한 del 사용
del s['부산']
s
```

```
Out[140... 서울    9731282
인천    2632035
대전    1490158
dtype: int64
```

3) 시리즈 데이터 추가

```
In [141... # 시리즈에 새로운 값 추가 : 딕셔너리의 방식과 동일
s['부산'] = 3393191
s['대구'] = 1875000
s
```

```
Out[141... 서울    9731282
인천    2632035
대전    1490158
부산    3393191
대구    1875000
dtype: int64
```

8. Series 관련 함수들

Series size, shape, unique, count, value_counts 함수

- size : 시리즈 원소 개수 반환
- shape : 튜플형태로 shape 반환
- unique(): 유일한 값만 ndarray로 반환
- count() : NaN을 제외한 개수를 반환
- mean() : NaN을 제외한 평균
- value_counts() : NaN을 제외하고 각 값들의 빈도를 반환

```
In [142... s = pd.Series([1,2,1,1,2,3,1,1,3,5,2,4,2,3,5, np.nan,5,3,2,1,4,2])
s
```

```
Out[142... 0      1.0
           1      2.0
           2      1.0
           3      1.0
           4      2.0
           5      3.0
           6      1.0
           7      1.0
           8      3.0
           9      5.0
          10      2.0
          11      4.0
          12      2.0
          13      3.0
          14      5.0
          15      NaN
          16      5.0
          17      3.0
          18      2.0
          19      1.0
          20      4.0
          21      2.0
dtype: float64
```

1) 시리즈 원소 크기

```
In [143... s.size
```

```
Out[143... 22
```

```
In [144... s.shape
```

```
Out[144... (22,)
```

```
In [145... len(s)
```

```
Out[145... 22
```

2) unique() : 시리즈의 유일한 값 반환

```
In [146... s.unique()
```

```
Out[146... array([ 1.,  2.,  3.,  5.,  4., nan])
```

```
In [148... set(s)
```

```
Out[148... {1.0, 2.0, 3.0, 4.0, 5.0, nan}
```

3) count() : NaN을 제외한 개수를 반환

```
In [147... s.count()
```

```
Out[147... np.int64(21)
```

4) mean() : 데이터의 평균

```
In [149... s.mean()
```

```
Out[149... np.float64(2.5238095238095237)
```

```
In [150... s.values.mean()
```

```
Out[150... np.float64(nan)
```

```
In [151... np.nanmean(s.values)
```

```
Out[151... np.float64(2.5238095238095237)
```

5) value_counts() : 그룹핑된 원소별 개수

```
In [154... s.unique()
```

```
Out[154... array([ 1.,  2.,  3.,  5.,  4., nan])
```

```
In [155... s.value_counts()
```

```
Out[155... 1.0    6
2.0    6
3.0    4
5.0    3
4.0    2
Name: count, dtype: int64
```

```
In [156... s.value_counts(dropna=False)
```

```
Out[156... 1.0    6
2.0    6
3.0    4
5.0    3
4.0    2
NaN     1
Name: count, dtype: int64
```

```
In [157... type(s.value_counts())
```

```
Out[157... pandas.Series
```

참고. 날짜 인덱스 생성

```
In [158... index_date = ['2024-12-07', '2024-12-08', '2024-12-09', '2024-12-10']
s = pd.Series(data=[200, 95, np.nan, 205], index=index_date)
s
```

```
Out[158... 2024-12-07    200.0
2024-12-08     95.0
2024-12-09      NaN
2024-12-10    205.0
dtype: float64
```

```
In [159... s.index
```

```
Out[159... Index(['2024-12-07', '2024-12-08', '2024-12-09', '2024-12-10'], dtype='str')
```

```
In [161... s.index > '2024-12-08'
```

```
Out[161... array([False, False,  True,  True])
```

날짜 생성 함수 : date_range()

- 판다스 패키지의 date_range 함수 (날짜생성)
- pd.date_range(start=None, end=None, periods=None, freq='D')
 - start : 시작날짜
 - end : 끝날짜
 - periods : 날짜 생성기간
 - freq : 날짜 생성 주기, 기본값은 'D' (Day)
 - start는 필수 옵션/end나 periods는 둘 중 하나가 있어야 함

freq 옵션

약어	설명	부가 설명 및 사용 예
D	달력 날짜 기준 하루 주기	하루 주기: freq = 'D', 이틀 주기: freq = '2D'
B	업무 날짜 기준 하루 주기	업무일(월요일 ~ 금요일) 기준으로 생성. freq = 'B', freq = '3B'
W	일요일 시작 기준 일주일 주기	월요일: W-MON, 화요일: W-TUE. freq = 'W', freq = 'W-MON'
M	월말 날짜 기준 주기	한 달 주기: freq = 'M', 네 달 주기: freq = '4M'
BM	업무 월말 날짜 기준 주기	freq = 'BM', freq = '2BM'
MS	월초 날짜 기준 주기	freq = 'MS', freq = '3MS'
BMS	업무 월초 날짜 기준 주기	freq = 'BMS', freq = '3BMS'
Q	분기 끝 날짜 기준 주기	freq = 'Q', freq = '2Q'
BQ	업무 분기 끝 날짜 기준 주기	freq = 'BQ', freq = '2BQ'
QS	분기 시작 날짜 기준 주기	freq = 'QS', freq = '2QS'
BQS	업무 분기 시작 날짜 기준 주기	freq = 'BQS', freq = '2BQS'
A	일년 끝 날짜 기준 주기	freq = 'A', freq = '5A'
BA	업무 일년 끝 날짜 기준 주기	freq = 'BA', freq = '3BA'
AS	일년 시작 날짜 기준 주기	freq = 'AS', freq = '2AS'
BAS	업무 일년 시작 날짜 기준 주기	freq = 'BAS', freq = '2BAS'
H	시간 기준 주기	1시간 주기: freq = 'H', 2시간 주기: freq = '2H'
BH	업무 시간 기준 주기	업무 시간 (09:00 ~ 17:00) 기준으로 생성
T, min	분 주기	10분 주기: freq = '10T', 30분 주기: freq = '30min'
S	초 주기	1초 주기: freq = 'S', 10초 주기: freq = '10S'

일별 간격으로 날짜 생성

```
In [164... idx = pd.date_range(start='2024-12-07', end='2024-12-31')
idx
```

```
Out[164... DatetimeIndex(['2024-12-07', '2024-12-08', '2024-12-09', '2024-12-10',
                '2024-12-11', '2024-12-12', '2024-12-13', '2024-12-14',
                '2024-12-15', '2024-12-16', '2024-12-17', '2024-12-18',
                '2024-12-19', '2024-12-20', '2024-12-21', '2024-12-22',
                '2024-12-23', '2024-12-24', '2024-12-25', '2024-12-26',
                '2024-12-27', '2024-12-28', '2024-12-29', '2024-12-30',
                '2024-12-31'],
                dtype='datetime64[us]', freq='D')
```

```
In [165... pd.date_range(start='2024-12-07', end='2024-12-31', freq='D')
```

```
Out[165... DatetimeIndex(['2024-12-07', '2024-12-08', '2024-12-09', '2024-12-10',
                '2024-12-11', '2024-12-12', '2024-12-13', '2024-12-14',
                '2024-12-15', '2024-12-16', '2024-12-17', '2024-12-18',
                '2024-12-19', '2024-12-20', '2024-12-21', '2024-12-22',
                '2024-12-23', '2024-12-24', '2024-12-25', '2024-12-26',
                '2024-12-27', '2024-12-28', '2024-12-29', '2024-12-30',
                '2024-12-31'],
                dtype='datetime64[us]', freq='D')
```

주간격으로 날짜 생성

- 지정한 일자가 포함된 주의 요일 시작 일자 기준으로 일주일 주기로 생성
- freq='W' : 일요일 기준으로 일주일 주기로 생성
- freq='W-MON' : 월요일 기준으로 일주일 주기로 생성

In [166... `pd.date_range(start='2024-12-07', end='2024-12-31', freq='W')`

Out[166... DatetimeIndex(['2024-12-08', '2024-12-15', '2024-12-22', '2024-12-29'], dtype='datetime64[us]', freq='W-SUN')

In [167... `pd.date_range(start='2024-12-07', end='2024-12-31', freq='W-MON')`

Out[167... DatetimeIndex(['2024-12-09', '2024-12-16', '2024-12-23', '2024-12-30'], dtype='datetime64[us]', freq='W-MON')

In [168... `pd.date_range(start='2024-12-07', end='2024-12-31', freq='W-TUE')`

Out[168... DatetimeIndex(['2024-12-10', '2024-12-17', '2024-12-24', '2024-12-31'], dtype='datetime64[us]', freq='W-TUE')

월간격으로 날짜 생성

- freq='ME' : 월말 날짜 기준 주기
- freq='MS' : 월초 날짜 기준 주기
- freq='BME' : 업무 월말 날짜 기준 주기
- freq='BMS' : 업무 월초 날짜 기준 주기

In [170... `pd.date_range(start='2023-12-07', end='2025-12-31', freq='ME')`

Out[170... DatetimeIndex(['2023-12-31', '2024-01-31', '2024-02-29', '2024-03-31', '2024-04-30', '2024-05-31', '2024-06-30', '2024-07-31', '2024-08-31', '2024-09-30', '2024-10-31', '2024-11-30', '2024-12-31', '2025-01-31', '2025-02-28', '2025-03-31', '2025-04-30', '2025-05-31', '2025-06-30', '2025-07-31', '2025-08-31', '2025-09-30', '2025-10-31', '2025-11-30', '2025-12-31'], dtype='datetime64[us]', freq='ME')

In [171... `pd.date_range(start='2023-12-07', end='2025-12-31', freq='MS')`

Out[171... DatetimeIndex(['2024-01-01', '2024-02-01', '2024-03-01', '2024-04-01', '2024-05-01', '2024-06-01', '2024-07-01', '2024-08-01', '2024-09-01', '2024-10-01', '2024-11-01', '2024-12-01', '2025-01-01', '2025-02-01', '2025-03-01', '2025-04-01', '2025-05-01', '2025-06-01', '2025-07-01', '2025-08-01', '2025-09-01', '2025-10-01', '2025-11-01', '2025-12-01'], dtype='datetime64[us]', freq='MS')

In [172... `pd.date_range(start='2023-12-07', end='2025-12-31', freq='BMS')`

Out[172... DatetimeIndex(['2024-01-01', '2024-02-01', '2024-03-01', '2024-04-01', '2024-05-01', '2024-06-03', '2024-07-01', '2024-08-01', '2024-09-02', '2024-10-01', '2024-11-01', '2024-12-02', '2025-01-01', '2025-02-03', '2025-03-03', '2025-04-01', '2025-05-01', '2025-06-02', '2025-07-01', '2025-08-01', '2025-09-01', '2025-10-01', '2025-11-03', '2025-12-01'], dtype='datetime64[us]', freq='BMS')

In [173... `pd.date_range(start='2023-12-07', end='2025-12-31', freq='BME')`

```
Out[173... DatetimeIndex(['2023-12-29', '2024-01-31', '2024-02-29', '2024-03-29',
                '2024-04-30', '2024-05-31', '2024-06-28', '2024-07-31',
                '2024-08-30', '2024-09-30', '2024-10-31', '2024-11-29',
                '2024-12-31', '2025-01-31', '2025-02-28', '2025-03-31',
                '2025-04-30', '2025-05-30', '2025-06-30', '2025-07-31',
                '2025-08-29', '2025-09-30', '2025-10-31', '2025-11-28',
                '2025-12-31'],
                dtype='datetime64[us]', freq='BME')
```

분기별 주기로 날짜 생성

- freq = QE : 분기 끝 날짜 기준 주기
- freq = QS : 분기 시작 날짜 기준 주기
- freq = BQE : 업무 분기 끝 날짜 기준 주기
- freq = BQS : 업무 분기 시작 날짜 기준 주기

```
In [174... pd.date_range(start='2023-12-07', end='2025-12-31', freq='QE')
```

```
Out[174... DatetimeIndex(['2023-12-31', '2024-03-31', '2024-06-30', '2024-09-30',
                '2024-12-31', '2025-03-31', '2025-06-30', '2025-09-30',
                '2025-12-31'],
                dtype='datetime64[us]', freq='QE-DEC')
```

```
In [175... pd.date_range(start='2023-12-07', end='2025-12-31', freq='QS')
```

```
Out[175... DatetimeIndex(['2024-01-01', '2024-04-01', '2024-07-01', '2024-10-01',
                '2025-01-01', '2025-04-01', '2025-07-01', '2025-10-01'],
                dtype='datetime64[us]', freq='QS-JAN')
```

```
In [176... pd.date_range(start='2023-12-07', end='2025-12-31', freq='BQS')
```

```
Out[176... DatetimeIndex(['2024-01-01', '2024-04-01', '2024-07-01', '2024-10-01',
                '2025-01-01', '2025-04-01', '2025-07-01', '2025-10-01'],
                dtype='datetime64[us]', freq='BQS-JAN')
```

```
In [177... pd.date_range(start='2023-12-07', end='2025-12-31', freq='BQE')
```

```
Out[177... DatetimeIndex(['2023-12-29', '2024-03-29', '2024-06-28', '2024-09-30',
                '2024-12-31', '2025-03-31', '2025-06-30', '2025-09-30',
                '2025-12-31'],
                dtype='datetime64[us]', freq='BQE-DEC')
```

연별 주기로 날짜 생성

- freq = YE : 일년 끝 날짜 기준 주기
- freq = YS : 일년 시작 날짜 기준 주기
- freq = BYE : 업무 일년 끝 날짜 기준 주기
- freq = BYS : 업무 일년 시작 날짜 기준 주기

```
In [178... pd.date_range(start='2023-12-07', end='2025-12-31', freq='YS')
```

```
Out[178... DatetimeIndex(['2024-01-01', '2025-01-01'], dtype='datetime64[us]', freq='YS-JAN')
```

```
In [179... pd.date_range(start='2023-12-07', end='2025-12-31', freq='YE')
```

```
Out[179... DatetimeIndex(['2023-12-31', '2024-12-31', '2025-12-31'], dtype='datetime64[us]', freq='YE-DEC')
```

```
In [180... pd.date_range(start='2023-12-07', end='2025-12-31', freq='BYS')
```

```
Out[180... DatetimeIndex(['2024-01-01', '2025-01-01'], dtype='datetime64[us]', freq='BYS-JAN')
```

```
In [181... pd.date_range(start='2023-12-07', end='2025-12-31', freq='BYE')
```

```
Out[181... DatetimeIndex(['2023-12-29', '2024-12-31', '2025-12-31'], dtype='datetime64[us]', freq='BYE-DEC')
```

시간 기준 주기로 날짜와 시간 생성

- freq = h : 시간 주기
- freq = bh : 업무 시간 주기
- freq = min : 분 주기
- freq = s : 초 주기

```
In [183... pd.date_range(start='2026-08-26', periods=10, freq='h')
```

```
Out[183... DatetimeIndex(['2026-08-26 00:00:00', '2026-08-26 01:00:00',  
                '2026-08-26 02:00:00', '2026-08-26 03:00:00',  
                '2026-08-26 04:00:00', '2026-08-26 05:00:00',  
                '2026-08-26 06:00:00', '2026-08-26 07:00:00',  
                '2026-08-26 08:00:00', '2026-08-26 09:00:00'],  
                dtype='datetime64[us]', freq='h')
```

```
In [184... pd.date_range(start='2026-08-26', periods=10, freq='bh')
```

```
Out[184... DatetimeIndex(['2026-08-26 09:00:00', '2026-08-26 10:00:00',  
                '2026-08-26 11:00:00', '2026-08-26 12:00:00',  
                '2026-08-26 13:00:00', '2026-08-26 14:00:00',  
                '2026-08-26 15:00:00', '2026-08-26 16:00:00',  
                '2026-08-27 09:00:00', '2026-08-27 10:00:00'],  
                dtype='datetime64[us]', freq='bh')
```

```
In [185... pd.date_range(start='2026-08-26', periods=10, freq='min')
```

```
Out[185... DatetimeIndex(['2026-08-26 00:00:00', '2026-08-26 00:01:00',  
                '2026-08-26 00:02:00', '2026-08-26 00:03:00',  
                '2026-08-26 00:04:00', '2026-08-26 00:05:00',  
                '2026-08-26 00:06:00', '2026-08-26 00:07:00',  
                '2026-08-26 00:08:00', '2026-08-26 00:09:00'],  
                dtype='datetime64[us]', freq='min')
```

```
In [186... pd.date_range(start='2026-08-26', periods=10, freq='s')
```

```
Out[186... DatetimeIndex(['2026-08-26 00:00:00', '2026-08-26 00:00:01',  
                '2026-08-26 00:00:02', '2026-08-26 00:00:03',  
                '2026-08-26 00:00:04', '2026-08-26 00:00:05',  
                '2026-08-26 00:00:06', '2026-08-26 00:00:07',  
                '2026-08-26 00:00:08', '2026-08-26 00:00:09'],  
                dtype='datetime64[us]', freq='s')
```
